

Ultimate Beginner Guide to Linux File Permissions

Understanding Linux permissions is the secret to stopping 90% of 'Permission Denied' errors before they happen. This guide breaks down the essential commands you need to manage file ownership and access safely every single day.

What you'll learn:

- 1 Deciphering the Permissions Code
- 2 Transferring File Ownership
- 3 The Secret Math of chmod
- 4 Protecting Sensitive Files

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Deciphering the Permissions Code

When you run 'ls -l', you see a string like '-rw-r----'. The first character is '-' for a file or 'd' for a directory. The next nine characters are broken into three groups of three: what the Owner can do, what the Group can do, and what everyone else (Others) can do. 'r' stands for read, 'w' for write, and 'x' for execute.

```
$ls -la /var/www/
```

Cloud & DevOps Daily

Linux File Permissions Explained (chmod, chown)

Tuesday, August 18, 2026

Swipe to tip 2 >> linkedin.com/in/mrmanrajsingh

2

TIP 2 OF 4

Transferring File Ownership

Every file in Linux belongs to a specific user and group. If a service like Apache or Nginx needs to update a log file or read a configuration, it must own it. You use 'chown' (change owner) followed by the new user, a colon, the new group, and the file name to safely hand over control.

```
$sudo chown webuser:webgroup config.json
```

Cloud & DevOps Daily

Linux File Permissions Explained (chmod, chown)

Tuesday, August 18, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

The Secret Math of chmod

Instead of letters, you can use numbers to set permissions. Read is 4, Write is 2, and Execute is 1. You add them up for each group. For example, Read (4) + Write (2) = 6. Doing this for Owner, Group, and Others gives you three numbers, like 644 (owner can read/write, others can only read).

```
$chmod 755 run-app.sh
```

Cloud & DevOps Daily

Linux File Permissions Explained (chmod, chown)

Tuesday, August 18, 2026

Swipe to tip 4 >> [linkedin.com/in/mramanrajsingh](https://www.linkedin.com/in/mramanrajsingh)

4

TIP 4 OF 4

Protecting Sensitive Files

SSH keys and passwords must be kept strictly private. Linux will actively block you from using an SSH private key if its permissions are too loose. Setting a file to '400' means only the owner can read it (4), and no one else has any access at all (00).

```
$chmod 400 ~/.ssh/id_rsa
```

Cloud & DevOps Daily

Linux File Permissions Explained (chmod, chown)

Tuesday, August 18, 2026

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

- + **r = Read (4):** Allows viewing file contents or listing directory files.
- + **w = Write (2):** Allows modifying file contents or adding/deleting files in a directory.
- + **x = Execute (1):** Allows running a file as a script/program or entering a directory.
- + **Owner** is the creator, **Group** is a collection of users, **Others** is everyone else.
- + Use `'chmod +x script.sh'` to quickly make a script runnable without math.

Watch Out For

- ! Running `'chmod 777'` on files to fix errors, which exposes sensitive data to everyone.
- ! Using `'chown -R'` on system directories like `/var` or `/etc`, which can break your entire OS.
- ! Forgetting that directory execution ('x') permission is required to enter or `'cd'` into it.
- ! Confusing the order of `user:group` in `chown` (it's always user first, then group).

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mrmanrajsingh